

Software Re-Engineering

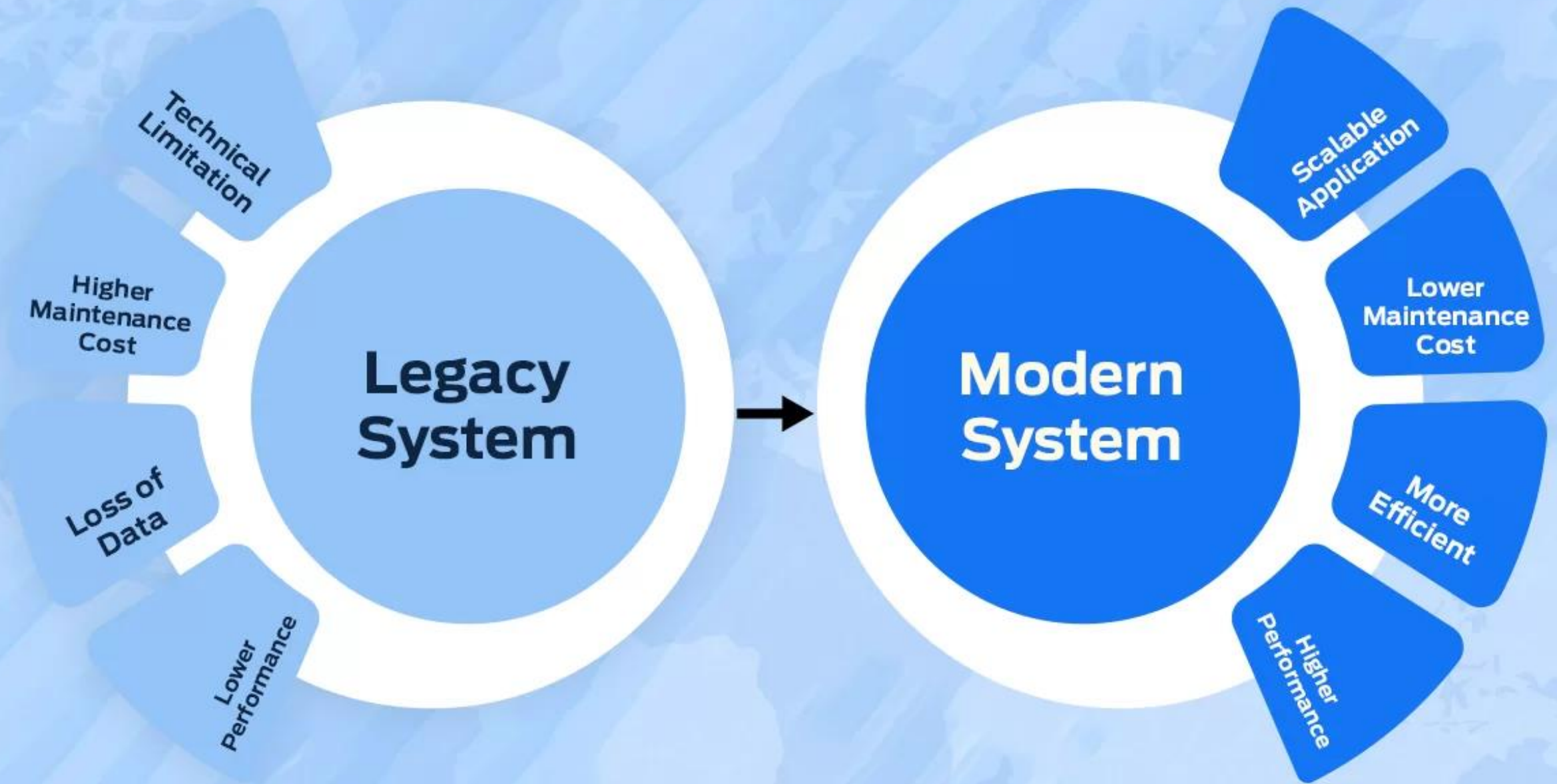
A thick, hand-drawn style orange line that spans approximately two-thirds of the width of the slide, positioned directly below the main title.

Week 6

Software Migration

Software Migration

- A process of transferring data, applications, or an entire software infrastructure from one environment to another.
- This process is essential in scenarios where organizations seek to upgrade their systems, shift to more scalable platforms, or adopt modern technologies for improved efficiency.
- Gartner reports show worldwide IT spending is expected to reach \$5.06 trillion in 2024, driven largely by cloud migration and modernization efforts.
 - An increase of 8% from 2023.



Types of Software Migration

Platform

Data

Application

Cloud

Database

Business
Process

Storage

Operating
System

Platform Migration

- It involves moving software or applications from one platform (hardware or operating system) to another. For example, an organization might migrate from Windows-based servers to Linux or from on-premises hardware to virtualized cloud platforms.
- **Why**
 - To enhance performance or compatibility with modern technology.
 - To reduce licensing or maintenance costs.
 - To take advantage of the scalability, flexibility, or security features of newer platforms.
- **Challenges**
 - Compatibility issues with the new platform.
 - Rewriting or adapting code to fit the new environment.

Data Migration

- It refers to the process of transferring data between storage systems, formats, or computer environments. This may involve consolidating data from multiple systems into a unified database or transferring data to a new storage system during a system upgrade.
- **Why**
 - To enable the use of more efficient or scalable storage solutions.
 - To improve data integrity, performance, and accessibility.
 - To merge multiple databases into one system, such as after a merger or acquisition.
- **Challenges**
 - Ensuring data integrity and consistency during the transfer.
 - Minimizing downtime during migration, as business operations rely on uninterrupted access to data.

Application Migration

- It involves moving an application from one environment to another, often requiring code modifications. This might be necessary if an organization wants to move an existing application to a new platform or cloud environment that doesn't support the original application architecture.
- **Why**
 - To update or modernize outdated applications.
 - To leverage cloud-native features (e.g., scaling, automation).
 - To increase the availability, performance, or security of the application.
- **Challenges**
 - Applications may need to be refactored or redesigned to be compatible with the new environment.
 - Downtime during migration and potential data loss are major concerns.

Cloud Migration

- It refers to moving software, data, or applications from on-premises infrastructure to cloud-based environments (such as AWS, Google Cloud, or Microsoft Azure). This is one of the most popular forms of migration due to the scalability and cost-effectiveness of cloud services.
- **Why**
 - To reduce infrastructure and maintenance costs by leveraging cloud services.
 - To improve system scalability, flexibility, and disaster recovery capabilities.
 - To access cloud-based applications and services that provide faster deployment and greater innovation.
- **Challenges**
 - Security concerns and compliance with regulations.
 - Ensuring that applications can function optimally in the cloud environment.
 - Managing the costs of cloud migration and ongoing cloud service fees.

Database Migration

- is the process of moving data from one database to another. This could involve upgrading to a newer version of the same database software or migrating to an entirely different database system.
- **Why**
 - To improve performance and take advantage of new features in modern databases.
 - To consolidate multiple databases into one centralized system.
 - To ensure compatibility with new or upgraded applications and systems.
- **Challenges**
 - Preserving data accuracy and integrity during migration.
 - Potential downtime or disruption to applications that rely on the database.
 - The need for data conversion if migrating to a database with a different structure or format.

Business Process Migration

- It focuses on transferring the business logic, rules, and workflows embedded in software or systems to a new setup. Often seen in enterprise resource planning (ERP) systems or when upgrading core business platforms.
- **Why**
 - To modernize outdated business process platforms and automate workflows.
 - To ensure the software aligns with evolving business needs and customer demands.
 - To integrate with other modern systems and improve efficiency.
- **Challenges**
 - Mapping and aligning complex business rules with new systems.
 - Ensuring that existing workflows are not disrupted during migration.
 - Managing change and ensuring staff are adequately trained on the new system.

Storage Migration

- It involves moving physical or virtual data storage to newer or more advanced solutions, such as from on-premise hard drives to cloud storage, or from one storage array to another.
- **Why**
 - To increase storage capacity and performance.
 - To take advantage of faster, more reliable storage technologies.
 - To meet security or regulatory requirements (e.g., migrating to encrypted storage).
- **Challenges**
 - Avoiding data loss or corruption during migration.
 - Ensuring that applications dependent on the storage system are not disrupted.

Operating System Migration

- It involves moving applications or entire systems from one operating system to another. This could involve upgrading an existing OS to a newer version or switching to a completely different OS (e.g., moving from Windows to Linux).
- **Why**
 - To take advantage of new features, security updates, or better performance.
 - To improve compatibility with certain applications or systems.
 - To reduce costs by moving to an open-source OS like Linux.
- **Challenges**
 - Ensuring that all applications are compatible with the new OS.
 - Potential need for retraining staff on the new system.
 - Downtime and transition issues if the migration is not well-planned.

Software Migration Comparison

Migration Type	Purpose	Challenges
Platform Migration	Move software to a new hardware/OS	Compatibility, downtime
Data Migration	Transfer data to new storage or system	Data integrity, downtime
Application Migration	Adapt an app to a new environment	Code refactoring, downtime
Cloud Migration	Move infrastructure to the cloud	Security, compliance, costs
Database Migration	Transfer data between databases	Data accuracy, downtime
Business Process Migration	Migrate business processes	Mapping workflows, training
Storage Migration	Upgrade storage systems	Data loss, performance
Operating System Migration	Move apps to a new OS	Compatibility, training

Strategies for Migrating Legacy Systems

Lift-and-Shift

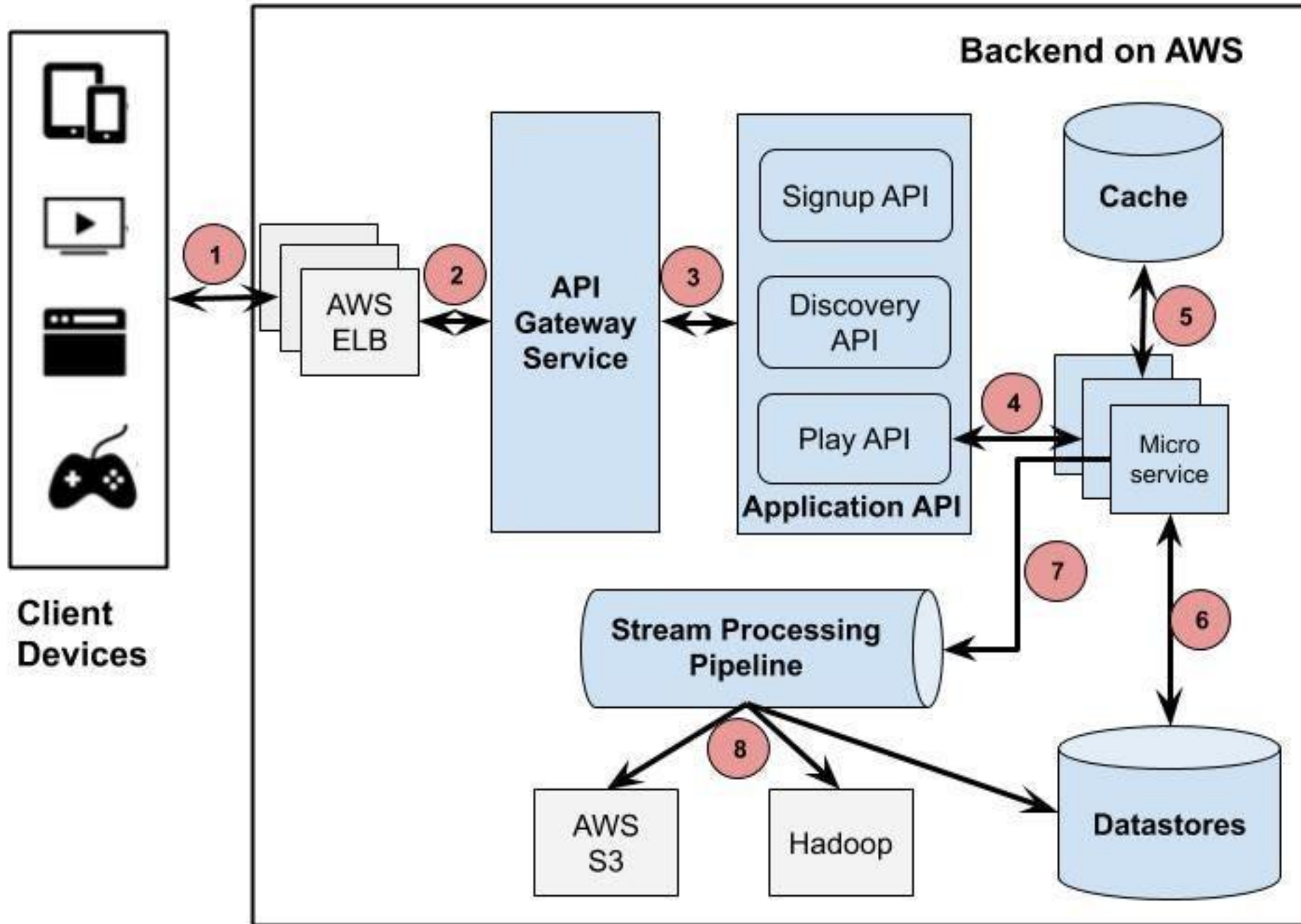
- This is the most straightforward method where an application is moved from one environment (e.g., on-premise) to another (e.g., cloud) without making any significant changes to its architecture. Often used when speed is the priority, and the legacy system is still functional.
 - Example: When Dropbox migrated their cloud storage to AWS to support their large-scale global operations.
- Use Cases
 - Moving data from local servers to AWS EC2 instances without altering the data structure.
- Pros
 - Minimal downtime.
 - Fast implementation.
 - No need for code refactoring.
- Cons
 - Doesn't pull cloud-native features.
 - Cost inefficiencies if not optimized post-migration.

Re-platforming

- This strategy involves optimizing certain aspects of the application to make it more efficient in the new environment, but without changing the core architecture.
 - Example: A database can be moved from an on-premises server to a managed cloud database service.
- **Use Cases**
 - Migrating from a custom-built database to a managed database solution like Amazon RDS.
- **Pros**
 - Moderate level of change, easier than complete refactoring.
 - Leverages some cloud-native features.
- **Cons**
 - Can require tuning and optimization after migration.
 - May involve some downtime.

Refactoring (Re-architecting)

- This involves a more in-depth process of rewriting or refactoring parts (or all) of the application to better fit the new system's architecture. This is typically done when the old system is inefficient or difficult to maintain.
 - **Example:** Netflix's move from a monolithic data center setup to a microservices architecture on AWS is a well-known example of refactoring.
- Use Cases
 - To move monolithic applications to microservices architecture (e.g., decomposing a large system into smaller, independently deployable services).
 - Legacy systems that no longer meet performance requirements.
- Pros
 - Takes full advantage of cloud-native features.
 - Better scalability, flexibility, and resilience.
- Cons
 - Time-consuming and expensive.
 - Higher technical risk if not managed properly.



Replacement

- In some cases, the most viable solution is to replace the legacy system altogether with a new solution that fits the organization's needs.
- Examples
 - Replacing an outdated CRM with Salesforce.
 - Replacing an on-premise ERP system with a cloud-based ERP like SAP S/4HANA.
- Use Case
 - When a legacy system is no longer maintainable, lacks support, or its architecture is incompatible with modern systems.
- Pros
 - Complete modernization and flexibility.
 - Alignment with the latest technologies.
- Cons
 - High cost and disruption.
 - Major retraining of users.

Challenges in Software Migration

Challenges in Software Migration

Compatibility
Issues

Data
Migration
Complexity

Downtime and
Service
Interruptions

User
Resistance

Legacy Code
Complexity

Security Risks

Compatibility Issues

- Legacy systems often run on old hardware or software platforms, which may not be compatible with modern operating systems or cloud environments.
- **Solution**
- Thorough assessment and mapping of legacy systems. Using compatibility layers.
 - E.g., Containers

Data Migration Complexity

- Transferring vast amounts of data, sometimes unstructured or poorly documented, poses a challenge. Data corruption and loss risks are high.
- **Solution**
- Develop a detailed data migration plan, perform data cleansing, and validate post-migration integrity.

Downtime and Service Interruptions

- Any migration brings the risk of downtime, which can lead to lost productivity or revenue.
- **Solution**
- Implement phased rollouts, fallback systems, and thorough testing.

User Resistance

- Resistance to change can slow down migration efforts.
- **Solution**
- Invest in training programs, communicate benefits early, and involve users in the migration process.

Legacy Code Complexity

- Legacy systems often contain complex codebases that are hard to understand or modify.
- **Solution**
- Use automated code analysis tools to assess code structure and identify critical parts that need refactoring.

Security Risks

- Migrating systems, especially cloud migrations, opens new attack paths.
- **Solution**
- Conduct security assessments and enforce encryption.



Questions?